

Logic Locking

Overview:

Prepare a report that documents your application of logic-locking and SAT-based attacks based on the problem described below.

Scenario: A 5-input, 2-output circuit, *c17*¹ has been previously locked by a manufacturer. You have been able to extract the logic-locked netlist. You now want to try and extract the key. An unlocked *c17* is available to you as a black box (which you can use as the oracle).

Retrieve the correct key of the encrypted *c17* using the SAT-attack, describing the steps that you take in your report. For this assignment, you will use the original *c17* netlist as your oracle.

“Unlock” your circuit using the SAT-attack.

- Algorithm 1 from this paper² could be helpful
- Your first step will be to construct the miter circuit
- You may use a SAT solver of your choice, but we would recommend <http://logictools.org/index.html>³

Document all the steps that you take in your report. Write out the formulas in each step (see the example).

¹ <http://www.pld.ttu.ee/~maksim/benchmarks/iscas85/bench/c17.bench>

² P. Subramanyan, S. Ray and S. Malik, "Evaluating the security of logic encryption algorithms," 2015 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), Washington, DC, 2015, pp. 137-143. doi: 10.1109/HST.2015.7140252, url: <https://ieeexplore.ieee.org/document/7140252>

³ NB: while SAT solvers typically expect circuits to be represented in Conjunctive Normal Form (CNF), you may describe your clauses as a propositional formula if you use <http://logictools.org/index.html> as your SAT solver. Logictools is also able to generate the CNF representation for you from the propositional formulas, so it's worth checking that out so that you understand CNF. Also, be sure to use **dp11:old** when trying to solve.

c17 encrypted netlist

```
INPUT(pi1)
INPUT(pi2)
INPUT(pi3)
INPUT(pi6)
INPUT(pi7)
INPUT(keyinput0)
INPUT(keyinput1)
INPUT(keyinput2)
INPUT(keyinput3)
OUTPUT(po22)
OUTPUT(po23$enc)
pi10 = nand(pi1, pi3)
pi11 = nand(pi3, pi6)
pi16 = and(pi2, pi11)
pi19 = and(pi11, pi7$enc)
po22 = nand(pi10, pi16$enc)
po23 = nand(pi16$enc, pi19$enc)
pi7$enc = xnor(keyinput0, pi7$inv)
pi7$inv = not(pi7)
pi16$enc = xnor(keyinput1, pi16)
pi19$enc = xor(keyinput2, pi19)
po23$enc = xor(keyinput3, po23)
```

c17 original netlist

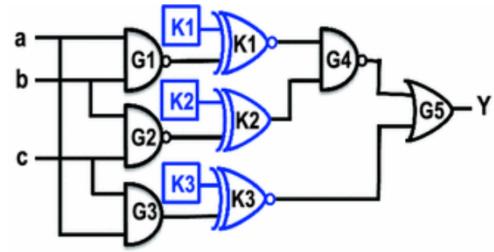
```
INPUT(pi1)
INPUT(pi2)
INPUT(pi3)
INPUT(pi6)
INPUT(pi7)

OUTPUT(po22)
OUTPUT(po23)

pi10 = NAND(pi1, pi3)
pi11 = NAND(pi3, pi6)
pi16 = NAND(pi2, pi11)
pi19 = NAND(pi11, pi7)
po22 = NAND(pi10, pi16)
po23 = NAND(pi16, pi19)
```

Example

Example circuit from “Hardware Security and Trust: Logic Locking as a Design-for-Trust Solution” by Muhammad Yasin, Bodhisatwa Mazumdar, Jeyavijayan Rajendran, Ozgur Sinanoglu



Oracle with correct key:

```
((~(a&b) <=> u0) &
(~(b&c) <=> u1) &
((c&a) <=> u2) &
(~(k1a+u0) <=> u5) &
((k2a + u1) <=> u4) &
(~(k3a+u2) <=> u3) &
(~(u5&u4) <=> u6) &
(y <=> (u6 | u3))) &
k1a &
~k2a &
k3a &
a &
b &
c
```

Here is the key

These are inputs you can change

This is useful instead of creating the truth table. In a “real” scenario, you might only have the physical circuit with you, to actual set inputs and get outputs...

Miter circuit:

```
((~(a&b) <=> u0) &
(~(b&c) <=> u1) &
((c&a) <=> u2) &
(~(k1a+u0) <=> u5) &
((k2a + u1) <=> u4) &
(~(k3a+u2) <=> u3) &
(~(u5&u4) <=> u6) &
(y <=> (u6 | u3))) &
((~(a&b) <=> w0) &
(~(b&c) <=> w1) &
((c&a) <=> w2) &
(~(k1b+w0) <=> w5) &
((k2b + w1) <=> w4) &
(~(k3b+w2) <=> w3) &
(~(u5&w4) <=> w6) &
(yx <=> (w6 | w3))) &
(y <=> ~yx) &
```

This is one copy of the circuit

Putting this in to logictool (dpll: old), we get:

Clause set is **true** if we assign values to variables as: u0 **a** -b u1 **c** u2 u5 k1a -u4 k2a u3 k3a u6 **y** w0 w1 w2 w5 k1b w4 -k2b -w3 -k3b -w6 -y2

Which tells us that we can use a=1, b=0, c=1 as a DIP. Using the DIP in the oracle tells us that y = 1 when the input is “101”. So, the next SAT formula to solve becomes:

```
((~(a&b) <=> u0) &
(~(b&c) <=> u1) &
((c&a) <=> u2) &
(~(k1a+u0) <=> u5) &
((k2a + u1) <=> u4) &
(~(k3a+u2) <=> u3) &
(~(u5&u4) <=> u6) &
(y <=> (u6 | u3))) &
((~(a1&b1) <=> n0) &
(~(b1&c1) <=> n1) &
((c1&a1) <=> n2) &
(~(k1a+n0) <=> n5) &
((k2a + n1) <=> n4) &
(~(k3a+n2) <=> n3) &
(~(n5&n4) <=> n6) &
(y1 <=> (n6 | n3))) &
((~(a1&b1) <=> m0) &
(~(b1&c1) <=> m1) &
((c1&a1) <=> m2) &
(~(k1b+m0) <=> m5) &
((k2b + m1) <=> m4) &
(~(k3b+m2) <=> m3) &
(~(m5&m4) <=> m6) &
(y1 <=> (m6 | m3))) &
```

a1 &
~b1 &
c1 &
y1

These are the new clauses, where we set
A1=1
B1=0
C1=1
Y1=1

In this way, you should be able to carry on, to solve for the key